Naman Uapure CS23093

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score,classification_report
```

```
from google.colab import drive
drive.mount('/content/drive')
path="/content/drive/MyDrive/ML Datasets/banking.csv"
df=pd.read_csv(path)
df.head(5)
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remou
```

	age	job	marital	education	default	housing	loan	contact	month	day_of_week	...	campaign	pdays	pr
0	44	blue-collar	married	basic.4y	unknown	yes	no	cellular	aug	thu	...	1	999	
1	53	technician	married	unknown	no	no	no	cellular	nov	fri	...	1	999	
2	28	management	single	university.degree	no	yes	no	cellular	jun	thu	...	3	6	
3	39	services	married	high.school	no	no	no	cellular	apr	fri	...	2	999	
4	55	retired	married	basic.4y	no	yes	no	cellular	aug	fri	...	1	3	

5 rows × 21 columns

```
import os

drive_root_path = '/content/drive/MyDrive/'

# Check if the directory exists first
if os.path.exists(drive_root_path):
    print(f"Files and directories in {drive_root_path}:")
    for filename in os.listdir(drive_root_path):
        print(filename)
else:
    print(f"The directory {drive_root_path} does not exist. This is unexpected after a successful drive mount.")
```

```
Files and directories in /content/drive/MyDrive/:
VIDEO
Classroom
Colab Notebooks
Automobile_data.gsheet
Untitled document.gdoc
Naman Eduskill Certificate.pdf
OS Code.pdf
OS Practical 01.pdf
OS Practical 02.pdf
OS Practical 03.pdf
OS Practical 04.pdf
OS Practical 05.pdf
OS Practical 06.pdf
OS Practical 07.pdf
OS Practical 08.pdf
OS Practical List.pdf
Naman Eduskill Certificate  (1).pdf
Naman Eduskill Certificate .pdf
CN Practical 06 (1).pdf
CN Practical 03 (1).pdf
CN Practical 07 (1).pdf
CN all practical Theory (1).pdf
CN Practical 04 (1).pdf
CN Practical 08 (1).pdf
CN Practical 12 (1).pdf
CN all practical codes (1).pdf
CN Practical 01 (1).pdf
CN Practical 02 (1).pdf
CN Practical 11 (1).pdf
CN Practical 05 (1).pdf
CN Practical 09 (1).pdf
CN Practical 10 (1).pdf
CN Practical 05.pdf
CN Practical 08.pdf
CN Postlab.pdf
CN Practical 10.pdf
CN all practical codes.pdf
CN Practical 07.pdf
CN Practical 01.pdf
CN Practical 06.pdf
CN Practical 03.pdf
CN Practical 09.pdf
```

```
CN Practical 12.pdf
CN Practical 02.pdf
CN all practical Theory.pdf
CN Practical 11.pdf
CN Practical 04.pdf
Naman Certificate for PTE.pdf
ML Datasets
```

```
import os

drive_ml_path = '/content/drive/MyDrive/ML/'

# Check if the directory exists first
if os.path.exists(drive_ml_path):
    print(f"Files in {drive_ml_path}:")
    for filename in os.listdir(drive_ml_path):
        print(filename)
else:
    print(f"The directory {drive_ml_path} does not exist. Please check the path.")
```

```
The directory /content/drive/MyDrive/ML/ does not exist. Please check the path.
```

```
df.shape
df.columns
```

```
Index(['age', 'job', 'marital', 'education', 'default', 'housing', 'loan',
       'contact', 'month', 'day_of_week', 'duration', 'campaign', 'pdays',
       'previous', 'poutcome', 'emp_var_rate', 'cons_price_idx',
       'cons_conf_idx', 'euribor3m', 'nr_employed', 'y'],
      dtype='object')
```

```
df.isnull().any().any()
```

```
np.False_
```

```
df[df.isnull().any(axis = 1)]
```

age	job	marital	education	default	housing	loan	contact	month	day_of_week	...	campaign	pdays	previous	pouto

0 rows × 21 columns

```
X = df.drop(columns=['y'])
y = df['y']
```

```
X.head()
```

	age	job	marital	education	default	housing	loan	contact	month	day_of_week	duration	campaign	pday
0	44	blue-collar	married	basic.4y	unknown	yes	no	cellular	aug	thu	210	1	99
1	53	technician	married	unknown	no	no	no	cellular	nov	fri	138	1	99
2	28	management	single	university.degree	no	yes	no	cellular	jun	thu	339	3	
3	39	services	married	high.school	no	no	no	cellular	apr	fri	185	2	99
4	55	retired	married	basic.4y	no	yes	no	cellular	aug	fri	137	1	

Next steps: Generate code with X New interactive sheet

```
X = pd.get_dummies(X,dtype = int)
```

```
X.head()
```

	age	duration	campaign	pdays	previous	emp_var_rate	cons_price_idx	cons_conf_idx	euribor3m	nr_employed	...	mo
0	44	210	1	999	0	1.4	93.444	-36.1	4.963	5228.1	...	
1	53	138	1	999	0	-0.1	93.200	-42.0	4.021	5195.8	...	
2	28	339	3	6	2	-1.7	94.055	-39.8	0.729	4991.6	...	
3	39	185	2	999	0	-1.8	93.075	-47.1	1.405	5099.1	...	
4	55	137	1	3	1	-2.9	92.201	-31.4	0.869	5076.2	...	

5 rows × 63 columns

```
X.shape
```

```
(41188, 63)
```

```
X.columns
```

```
Index(['age', 'duration', 'campaign', 'pdays', 'previous', 'emp_var_rate',
       'cons_price_idx', 'cons_conf_idx', 'euribor3m', 'nr_employed',
       'job_admin.', 'job_blue-collar', 'job_entrepreneur', 'job_housemaid',
       'job_management', 'job_retired', 'job_self-employed', 'job_services',
       'job_student', 'job_technician', 'job_unemployed', 'job_unknown',
       'marital_divorced', 'marital_married', 'marital_single',
       'marital_unknown', 'education_basic.4y', 'education_basic.6y',
       'education_basic.9y', 'education_high.school', 'education_illiterate',
       'education_professional.course', 'education_university.degree',
       'education_unknown', 'default_no', 'default_unknown', 'default_yes',
       'housing_no', 'housing_unknown', 'housing_yes', 'loan_no',
       'loan_unknown', 'loan_yes', 'contact_cellular', 'contact_telephone',
       'month_apr', 'month_aug', 'month_dec', 'month_jul', 'month_jun',
       'month_mar', 'month_may', 'month_nov', 'month_oct', 'month_sep',
       'day_of_week_fri', 'day_of_week_mon', 'day_of_week_thu',
       'day_of_week_tue', 'day_of_week_wed', 'poutcome_failure',
       'poutcome_nonexistent', 'poutcome_success'],
      dtype='object')
```

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,y,train_size=0.7,test_size=0.3,random_state=42)
X_train.shape,X_test.shape
from sklearn.ensemble import RandomForestClassifier
rf_classifier = RandomForestClassifier(n_estimators=100,n_jobs=-1,max_depth=5,oob_score=True,random_state=42)
rf_classifier.fit(X_train,y_train)
```

```
RandomForestClassifier
RandomForestClassifier(max_depth=5, n_jobs=-1, oob_score=True, random_state=42)
```

```
y_pred = rf_classifier.predict(X_test)
#print(y_pred)
```

```
from sklearn.metrics import confusion_matrix,accuracy_score,classification_report
```

```
print('Confusion Matrix')
cm=confusion_matrix(y_test,y_pred)
print(confusion_matrix(y_test,y_pred))
```

```
Confusion Matrix
[[10848    83]
 [ 1158   268]]
```

```
print('Accuracy Score')
print(accuracy_score(y_test,y_pred))
```

```
Accuracy Score
0.8995710933074371
```

```
rf_classifier.oob_score_
```

```
0.9024660955221809
```

```
print('Classification Report')
print(classification_report(y_test,y_pred))
```

```
Classification Report
              precision    recall  f1-score   support

           0       0.90      0.99      0.95     10931
           1       0.76      0.19      0.30      1426

    accuracy                           0.90     12357
   macro avg       0.83      0.59      0.62     12357
weighted avg       0.89      0.90      0.87     12357
```